

FADO: Floorplan-Aware Directive Optimization for HLS Designs on Multi-die FPGAs

Linfeng Du¹, Tingyuan Liang¹, Sharad Sinha², Zhiyao Xie¹, Wei Zhang¹

Hong Kong University of Science and Technology¹, Indian Institute of Technology Goa²

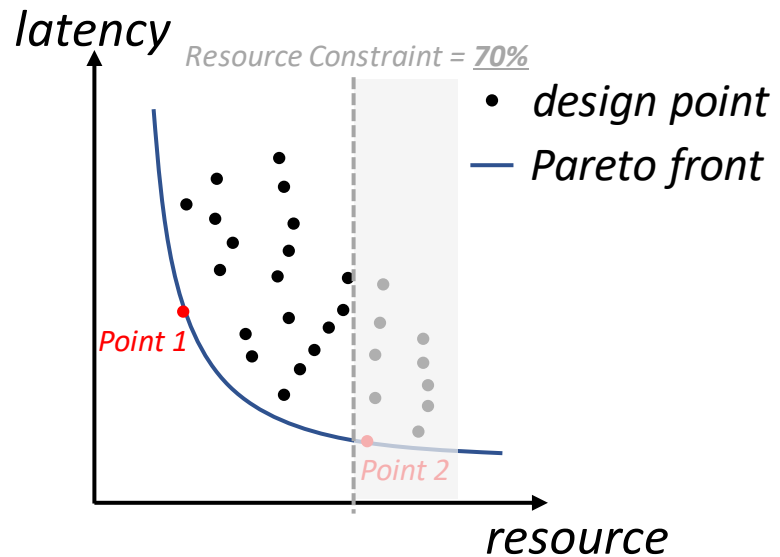
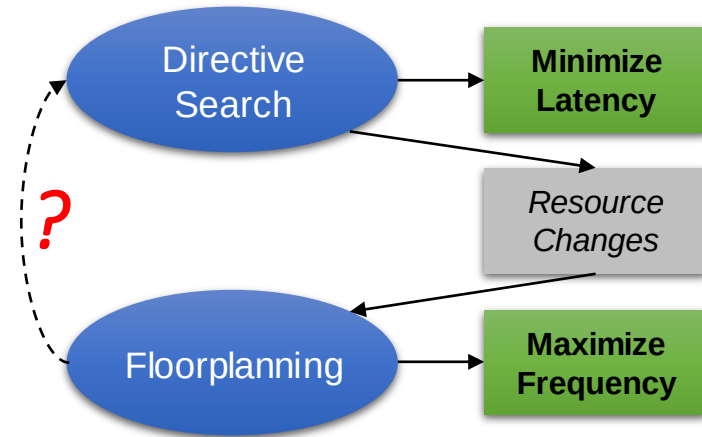
linfeng.du@connect.ust.hk

Open Source: <https://github.com/RipperJ/FADO>



Problem

- Co-optimization of
 - a) High-level synthesis (HLS) directive/pragma
 - b) Floorplan (on multi-die FPGA)



Point 1 – Under Utilization

```
void top(int A[N], int B[N], int C[N]) {  
    for (int i = 0; i < N; i++) {  
        #pragma HLS unroll factor=2  
        ...  
    }  
}
```

Latency: 1000 cycles
Resource: 15%
Fmax: 270 MHz

Point 2 – Over Utilization

```
void top(int A[N], int B[N], int C[N]) {  
    for (int i = 0; i < N; i++) {  
        #pragma HLS unroll factor=16  
        ...  
    }  
}
```

Latency: 130 cycles ↓
Resource: 80% (> 70%) ↑
(FF, LUT, DSP, ...)
Fmax: 95 MHz ↓

Challenges in Directive Search

1. Large design space

- Altogether 26 directives (Vitis HLS 2020.2)
- Various parameters

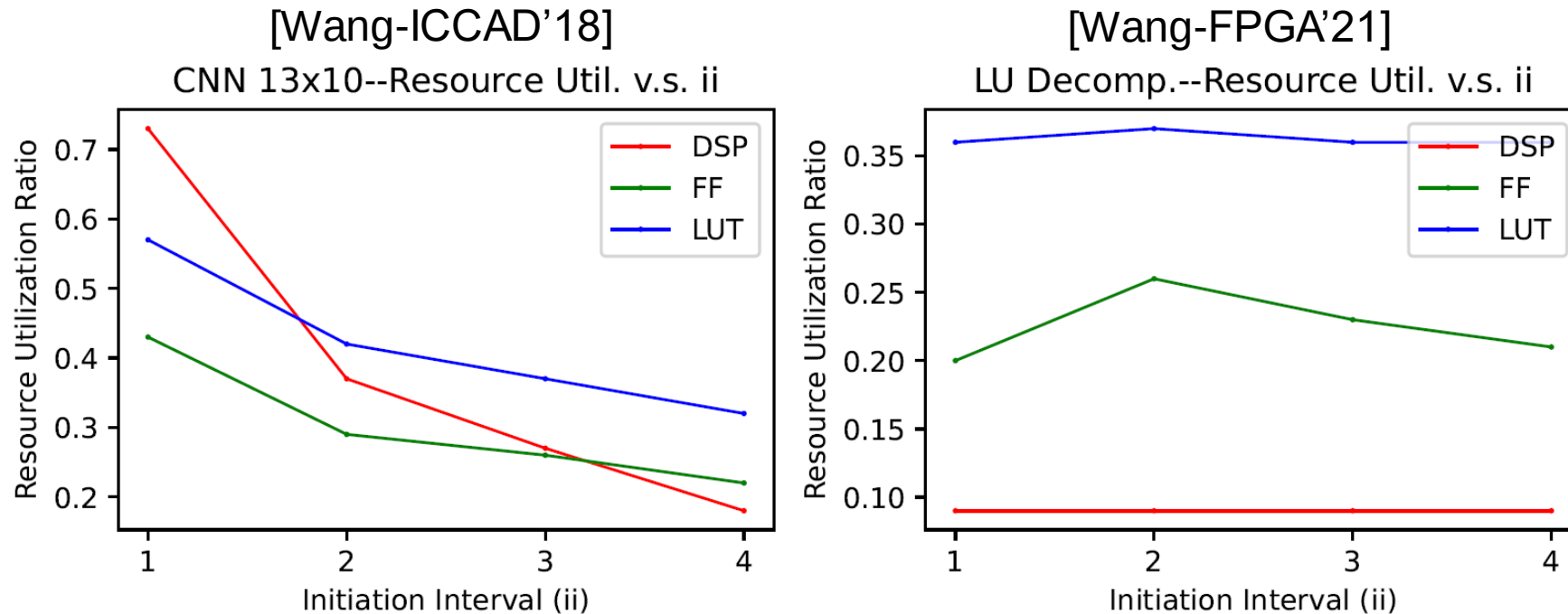
Directives and Parameters in the Design Space of FADO

Directives	Parameters
PIPELINE	Initiation Interval (II) ($\langle \text{int} \rangle$: $\{MinII, \dots, \lfloor 4 \times MinII, IterLat \rfloor\}$)
UNROLL	Factor ($\langle \text{int} \rangle$: $\{1, 2, 4, \dots, LoopBound\}$)
ARRAY_PARTITION	Type (Block/Cyclic/Complete) Dimension ($\langle \text{int} \rangle$)
BIND_STORAGE	Implementation (BRAM/URAM)

- A highly efficient DSE algorithm is needed.

Challenges in Directive Search

2. Non-monotonic relationship between directives and QoRs



Comparison of Resource Utilization v.s. Initiation Interval (ii) between CNN and LU Benchmarks

- How to avoid getting trapped in local optima?

Challenges in Directive Search

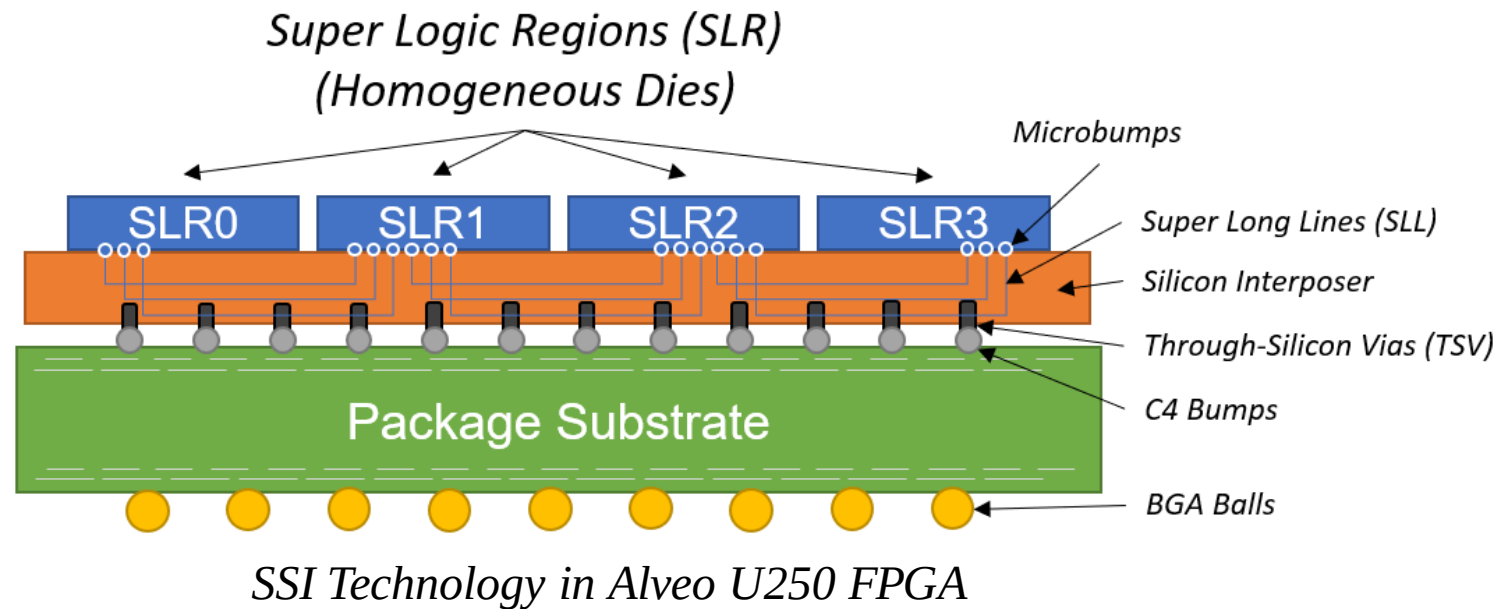


3. Previous solutions only consider overall resource of a single die.

- Meta heuristics
 - [Schafer-VLSI'09, Schafer-2017, Xydis-VLSI'10, ...]
 - Dedicated heuristics
 - [Pham-DATE'15, Piccolboni-TECS'17, Yu-DAC'18, ...]
 - Machine learning
 - [Meng-DATE'16, Wang-DATE'20, Zacharopoulos-HPCS'18, ...]
 - Graph analysis
 - [Sohrabizadeh-2021, Wu-VLSI'21, Ye-HPCA'22, Zhao-ICCAD'17, ...]
-
- How to consider separate constraints on/between multiple dies?

Challenges in Floorplanning

- Long net delay penalty
 - Inter-die (SLL): $\sim 1\text{ns}$ [Chaware-ECTC'12]
 - Intra-die: $125\text{ps} \sim 250\text{ps}$ [Nasiri-VLSI'15]



Challenges in Floorplanning

- Fine-grained (primitives/cells → tile/site/BEL)
- Coarse-grained (module/function → SLR/FSR)

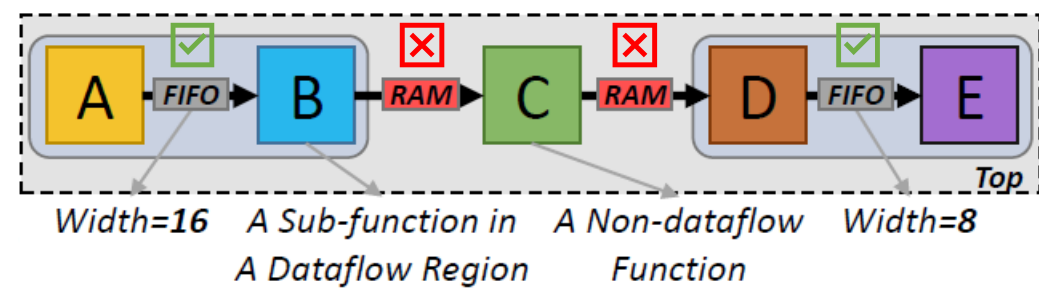
Work	Objectives					Algorithm/Tool	Granularity
	Total Wirelength	Signal Delay	Cut #	Routing Congestion	Aspect Ratio		
Mao-GLSVLSI'16	✓	✓				Force-directed --> SA	Coarse
Pereira-FPGA'14 Nasiri-VLSI'15		✓	✓		✓	VPR	Fine
Ravishankar-FPL'18			✓	✓	✓	Partition-based	Fine
Alonso-TRETS'21				✓		ILP (Balancing)	Coarse (HLS)
Guo-FPGA'21			✓			ILP (Min-cut)	Coarse (HLS)

- Faster floorplanning algorithm to integrate with directive search
- How to handle non-dataflow modules?

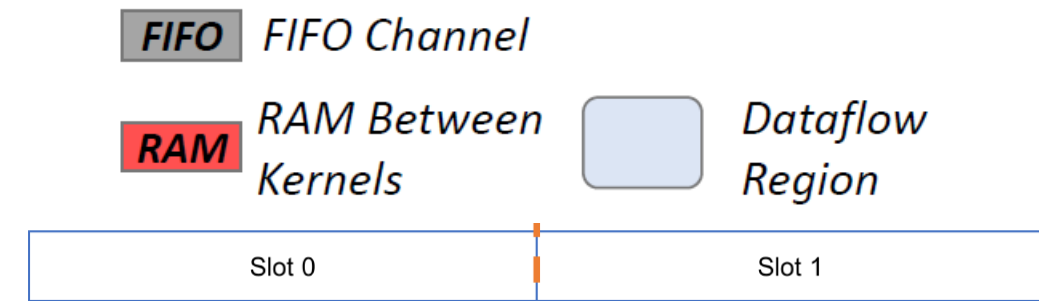
Motivation Example

- A toy design
 - 2 dataflow modules (regions)
 - 1 non-dataflow module
 - FIFO between sub-modules within the same dataflow region
 - BRAM between modules
 - At the start of DSE, all function has no directive applied.

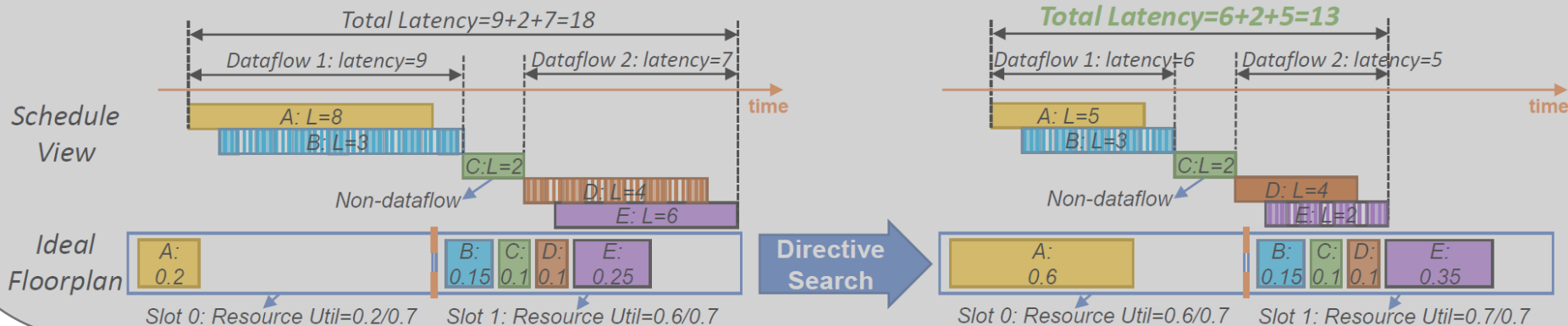
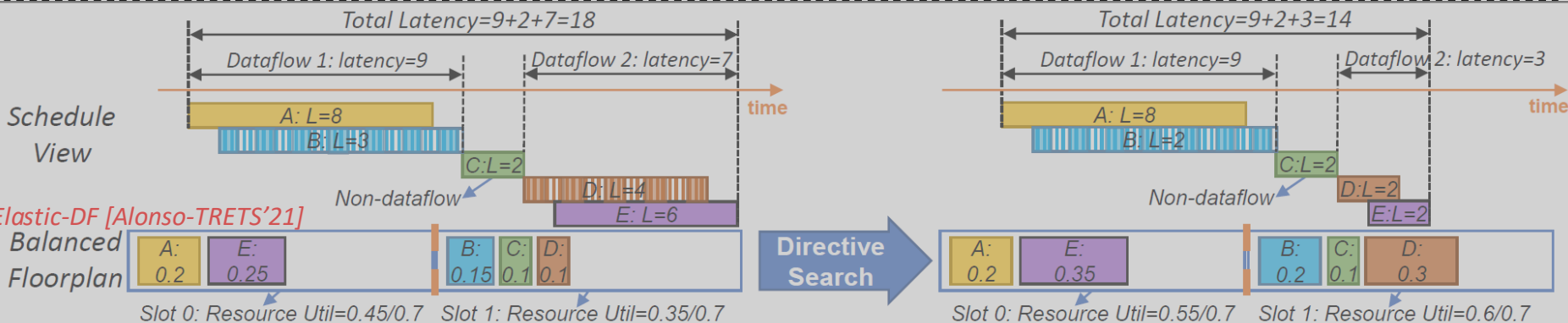
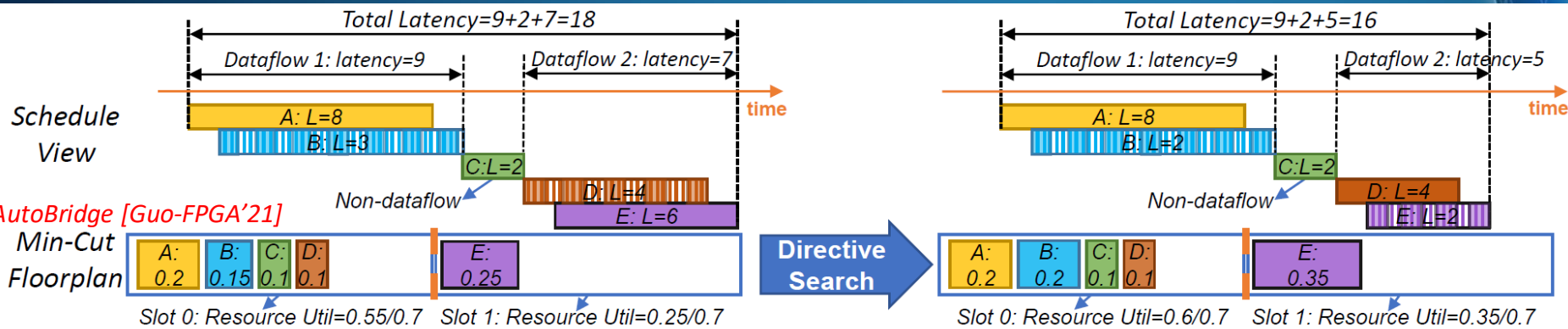
HLS Functions		Without Directive		With Directive	
		Resource	Latency	Resource	Latency
A		0.2	8	0.6	5
B		0.15	3	0.2	2
C		0.1	2	0.3	1
D		0.1	4	0.3	2
E		0.25	6	0.35	2



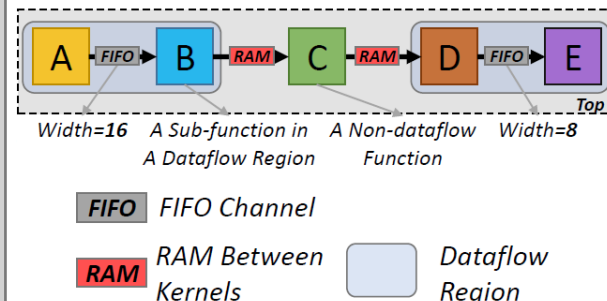
- A toy multi-die FPGA
 - 2 slots
 - Resource constraint: 70%



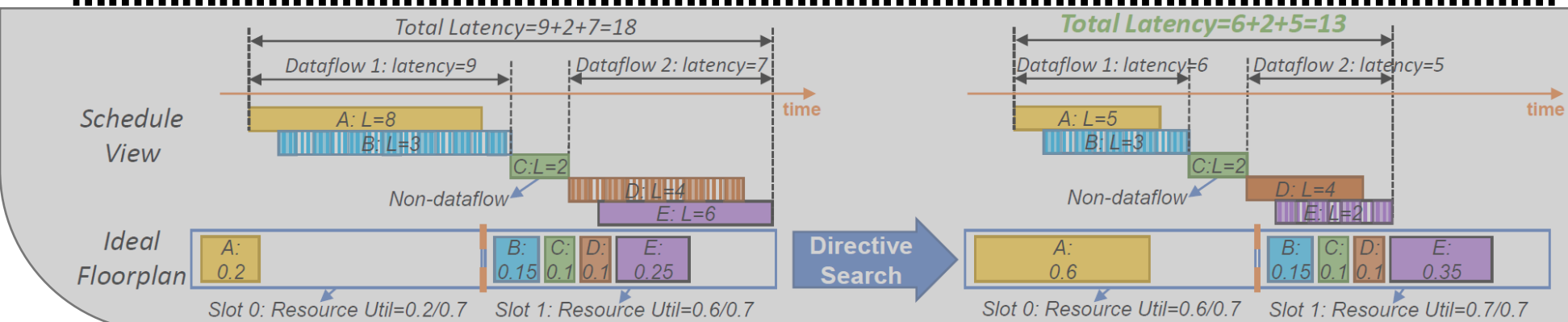
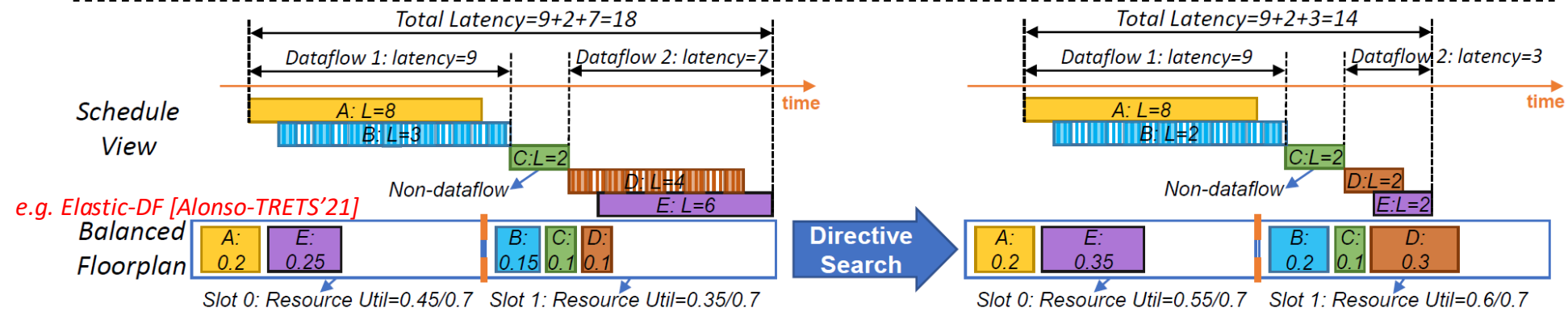
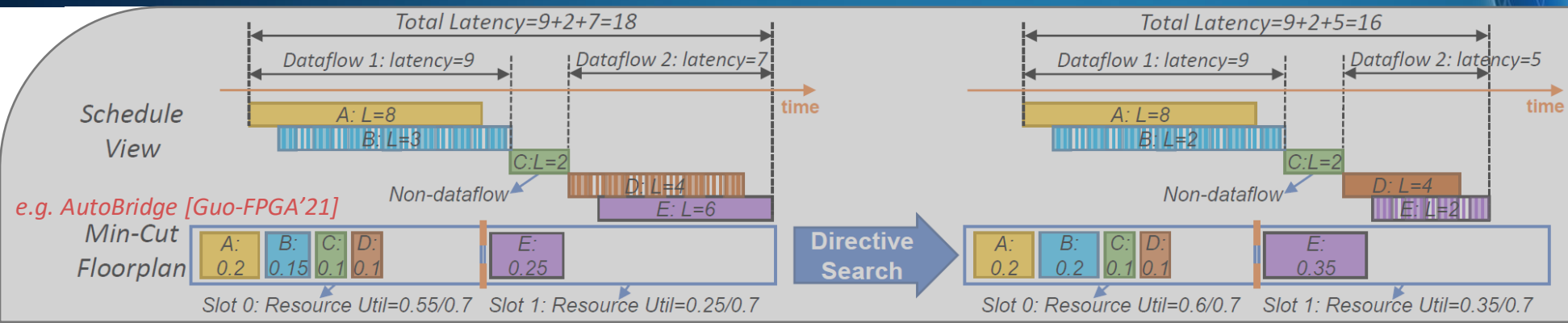
Motivation Example



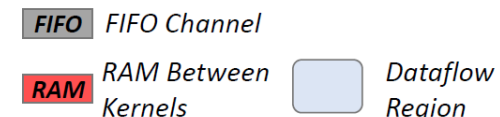
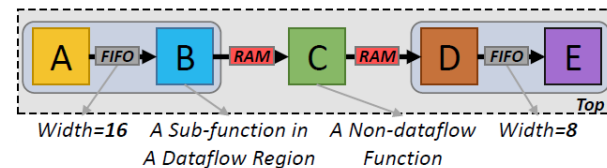
HLS Functions	Without Directive		With Directive	
	Resource	Latency	Resource	Latency
A	0.2	8	0.6	5
B	0.15	3	0.2	2
C	0.1	2	0.3	1
D	0.1	4	0.3	2
E	0.25	6	0.35	2



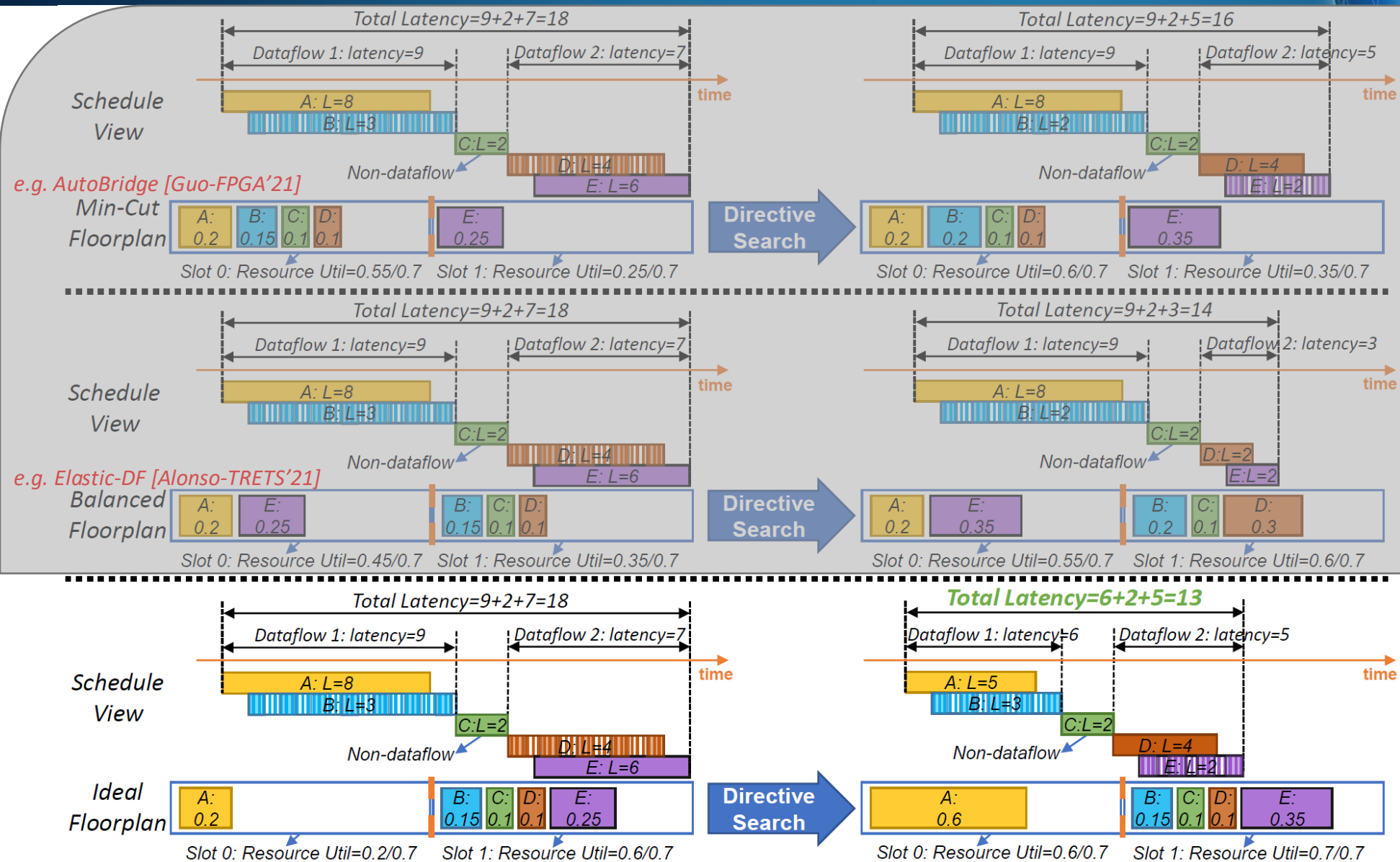
Motivation Example



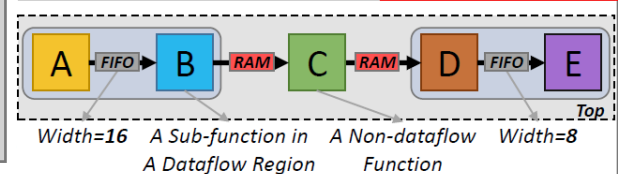
HLS Functions	Without Directive		With Directive	
	Resource	Latency	Resource	Latency
A	0.2	8	0.6	5
B	0.15	3	0.2	2
C	0.1	2	0.3	1
D	0.1	4	0.3	2
E	0.25	6	0.35	2



Motivation Example



HLS Functions	Without Directive		With Directive	
	Resource	Latency	Resource	Latency
A	0.2	8	0.6	5
B	0.15	3	0.2	2
C	0.1	2	0.3	1
D	0.1	4	0.3	2
E	0.25	6	0.35	2



FIFO FIFO Channel

RAM RAM Between Kernels

Dataflow Region

Comparison with Previous Work

	Directive Search	Multi-die Floorplanning	Floorplan-aware Directive DSE (FADO)
QoR	latency, resource	timing (frequency)	latency, resource, timing
Design Space	1. directives & parameters	2. SLR-level func location	1 & 2
DSE Efficiency	syn: slow; model: fast	slow (SA, MILP, bi-partition, ...)	fast (incremental floorplanning)
Type of Benchmark	dataflow or non-dataflow	dataflow	mixed dataflow & non-dataflow

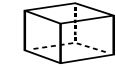
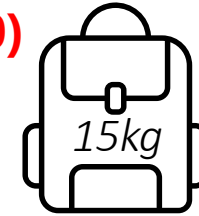
Problem Formulation

- Knapsack/Bin-packing Problem

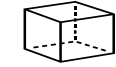
- (0) 0-1 **Knapsack** problem:

- Maximize value (-latency), s.t.: weight (resource)
 - Items – HLS functions; Knapsack: a slot/die

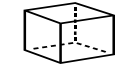
(0)



\$4, 9kg



\$7, 5kg



\$10, 14kg

Problem Formulation

- Knapsack/Bin-packing Problem

- (0) 0-1 **Knapsack** problem:

- Maximize value (-latency), s.t.: weight (resource)

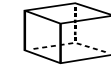
- Items – HLS functions; Knapsack: a slot/die

- (1) Multi-choice **Knapsack** problem:

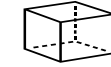
- pick 1 item from each class

- Multiple directives/parameters for each HLS function

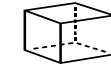
(0)



\$4, 9kg

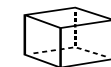
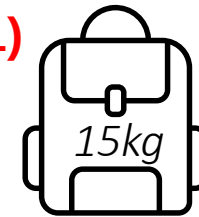


\$7, 5kg

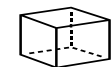


\$10, 14kg

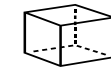
(1)



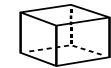
\$4, 9kg



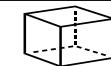
\$2, 4kg ... ^{Class 1}



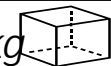
\$7, 5kg



\$8, 6kg ... ^{Class 2}



\$10, 14kg



\$7, 8kg ... ^{Class 3}

Problem Formulation

- Knapsack/Bin-packing Problem

- (0) 0-1 **Knapsack** problem:

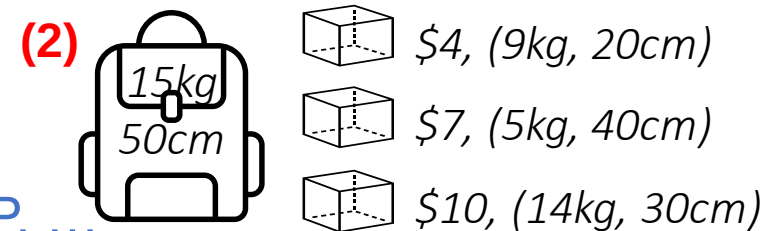
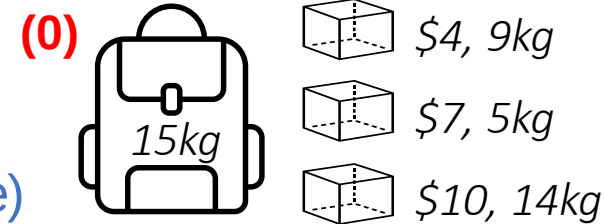
- Maximize value (-latency), s.t.: weight (resource)
 - Items – HLS functions; Knapsack: a slot/die

- (1) Multi-choice **Knapsack** problem:

- pick 1 item from each class
 - Multiple directives/parameters for each HLS function

- (2) Multi-dimensional **Knapsack** problem

- Multiple constraints, e.g., weight & length
 - Multiple resources on FPGA, e.g., FF, LUT, DSP, ...



Problem Formulation

- Knapsack/Bin-packing Problem

- (0) 0-1 **Knapsack** problem:

- Maximize value (-latency), s.t.: weight (resource)
 - Items – HLS functions; Knapsack: a slot/die

- (1) Multi-choice **Knapsack** problem:

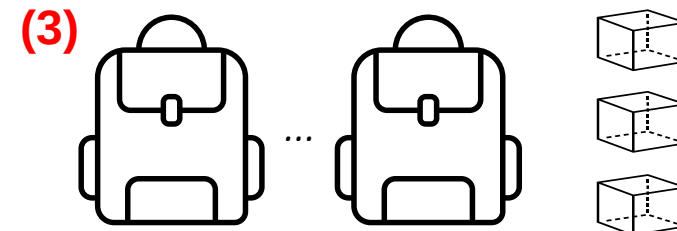
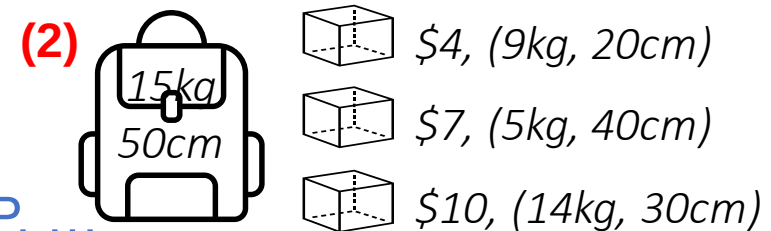
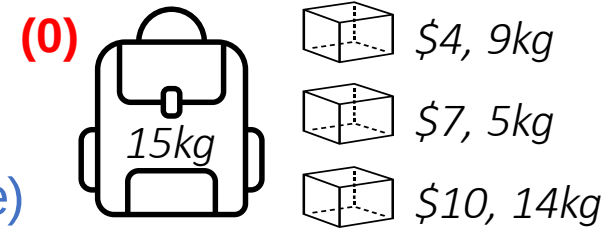
- pick 1 item from each class
 - Multiple directives/parameters for each HLS function

- (2) Multi-dimensional **Knapsack** problem

- Multiple constraints, e.g., weight & length
 - Multiple resources on FPGA, e.g., FF, LUT, DSP, ...

- (3) **Bin-packing** problem

- Multiple bins; All items must be packed
 - Legal floorplan found for all functions.



Problem Formulation

- Given an application mixing dataflow/non-dataflow kernels

- Objective: to minimize the total latency

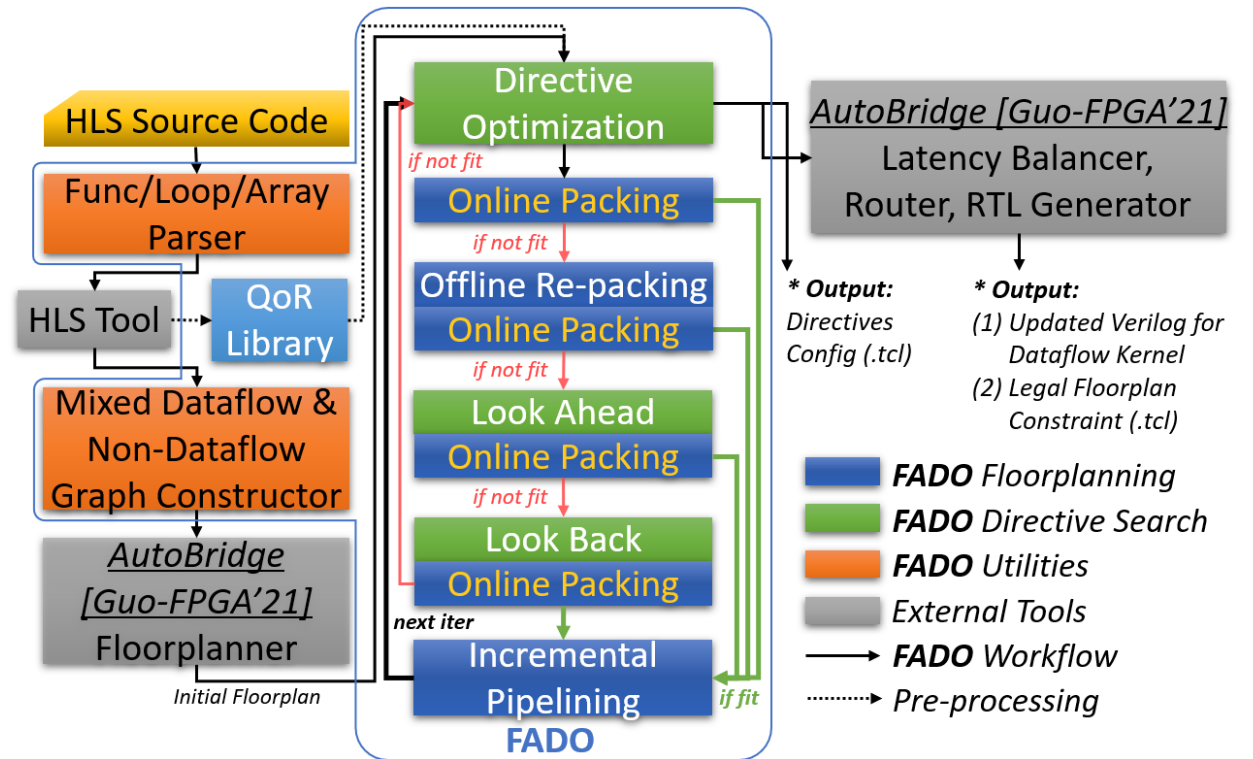
- Constraints:

Multi-choice Multi-dimensional Bin-packing Problem

- Multiple directives/parameters for each HLS function – multi-choice
- Multiple types of resources (FF, LUT, DSP, BRAM, URAM) – multi-dimensional
- Slots/Dies in floorplanning – bins
- BRAM-connected functions are grouped
- Die-crossing SLL number limit

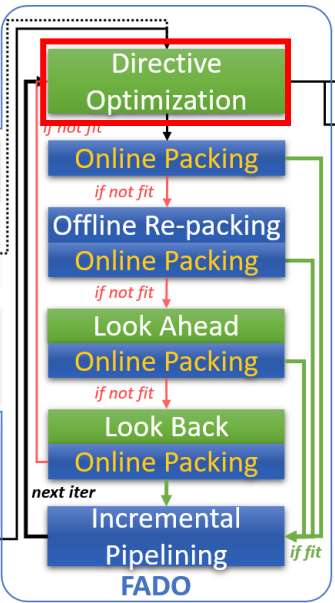
Implementation

- Floorplan-Aware Directive Optimization
 - Directive Optimization
 - Incremental Floorplanning
 - Incremental Pipelining



Implementation

- Directive Optimization
 - Main Iteration:
 - Guided by the latency bottleneck module
 - Optimize the longest module towards the 2nd longest module in each iteration
 1. $Latency_{DP} < L_{(i_2, j_2)}$
 2. $Latency_{DP} = \max(Latency_{DS[k]})$



Algorithm 1: Top-level of FADO framework

Input: QoR_lib , N : look-ahead step number

Output: Optimal Directives, Legal Floorplan

```
1 while True do
2    $(i_1, j_1) = \text{argmax}_{i,j}^1(L_{ij})$            // the longest functions
3    $L_{(i_2, j_2)} = \max_{i,j}^2(L_{ij})$            // the 2nd-longest latency
4    $DS = \text{Prune}(QoR\_lib, (imax,1, jmax,1), Li_{imax,2, jmax,2})$ ;
5    $DP = DS[-1]$ , break if  $DP$  is None // Constraint Eq. 3
6    $fit, incrfp\_list = \text{online\_packing}((i_1, j_1), DP)$ ;
7   if not fit then
8      $\text{offline\_repacking}, \text{online\_packing}$ 
9     if not fit then
10       $\text{iterative look\_ahead}(N), \text{online\_packing}$ 
11      if not fit then
12         $\text{iterative look\_back}(), \text{online\_packing}$ 
13   $\text{incremental\_Floorplan}(incrfp\_list)$ ;
14   $\text{exit\_condition\_check}()$ ;
```

- Directive Optimization
 - Sampling strategy to avoid local optima

- Ahead:

$$N_1 = \max_{1 \leq i \leq m} \sum_{j=1}^{\max(3, n_i)} \log_2 \min(64, IL_{ij})$$

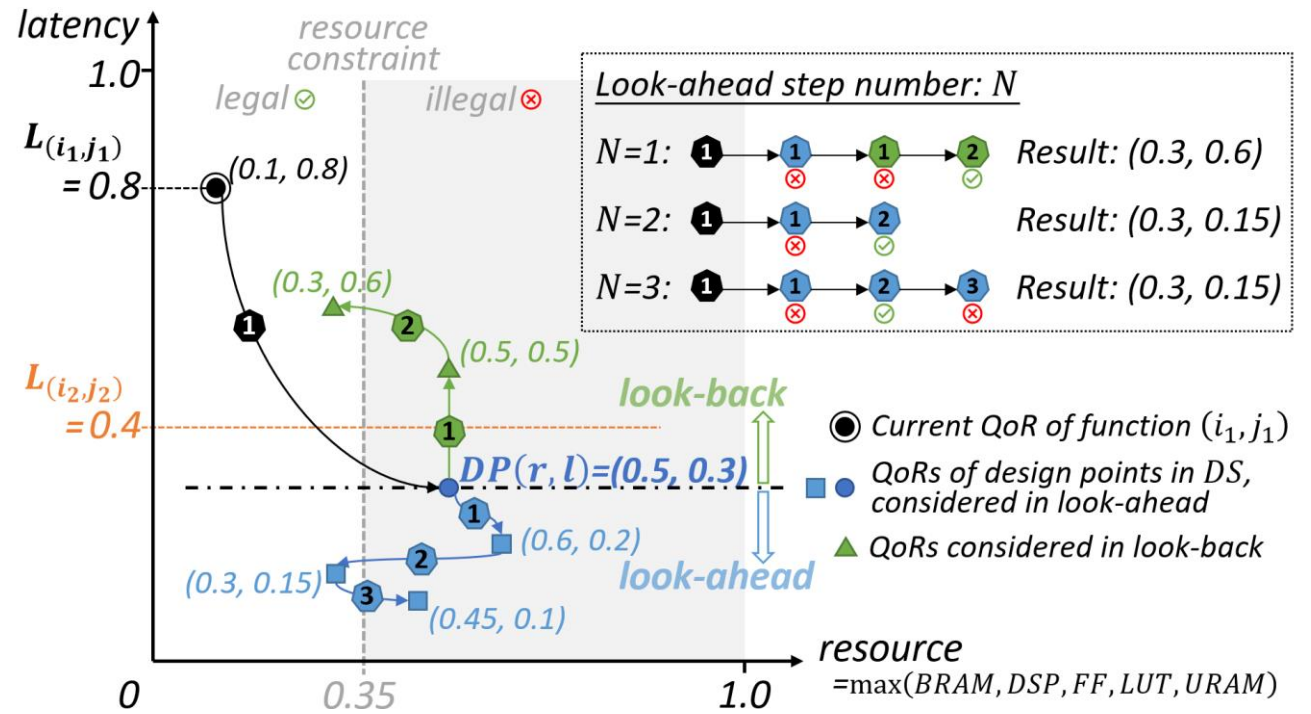
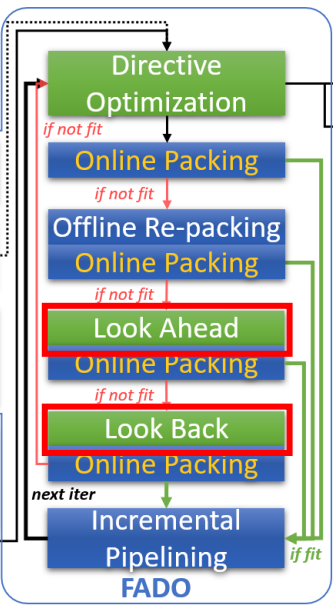
$$N_2 = \max_{1 \leq i \leq m} \sum_{j=1}^{\max(2, n_i)} \log_2 \min(64, B_{ij})$$

$$\begin{aligned} N_3 &= \max_{1 \leq i \leq m} \sum_{j=1}^n \log_2 \min(64, B_{ij}) \\ N &= N_1 + N_2 + N_3 \end{aligned}$$

$$N = N_1 + N_2 + N_3$$

- Back:

- Till the longest latency possible



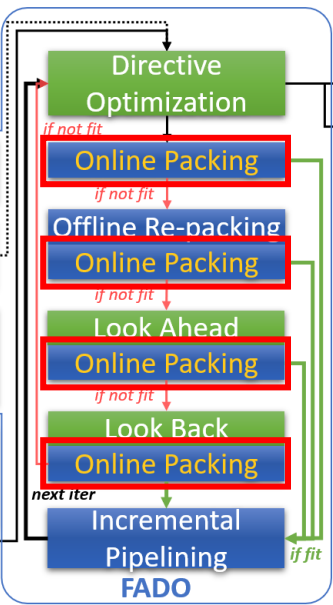
Effect of Look-Ahead with Step # N , and Look-Back.

Implementation

- Incremental Floorplanning

- Online Packing

- Worst-Fit heuristics



Algorithm 2: Online Packing

Input: *longest_functions* and QoRs L_{ij}, R_{ij} , design point p

Output: *fit*, *incrfp_list*

```
1 fit = False, incrfp_list = [], unfit_funcs = [];
2 for func in longest_functions do
3   if func still fits in the current slot  $s_c$  then
4     update directives for func, and resource util. for  $s_c$ ;
5      $L_{ij}, R_{ij} = L_p, R_p$ ; fit = True;
6   else
7     // check constraint Eq. 6
8     check SLL util.; // constraint Eq. 7 and Eq. 8
9     calculate overflow ratio;
10    sort other_slots by CR;
11    for each  $s_o$  in other_slots do
12      if no overflow when moving func to  $s_o$  then
13        // check constraint Eq. 5
14        update directives for func;
15        update util. for  $s_o, s_c$ ; // constraint Eq. 4
16        append (func,  $s_o$ ) to incrfp_list;
17         $L_{ij}, R_{ij} = L_p, R_p$ ; fit = True;
18        break
19    if not fit then
20      append func to unfit_funcs;
21 if any func in unfit_funcs then
22   clear incrfp_list; fit = False;
```

Apply new directives

Update resource util.

Check resource util. overflow

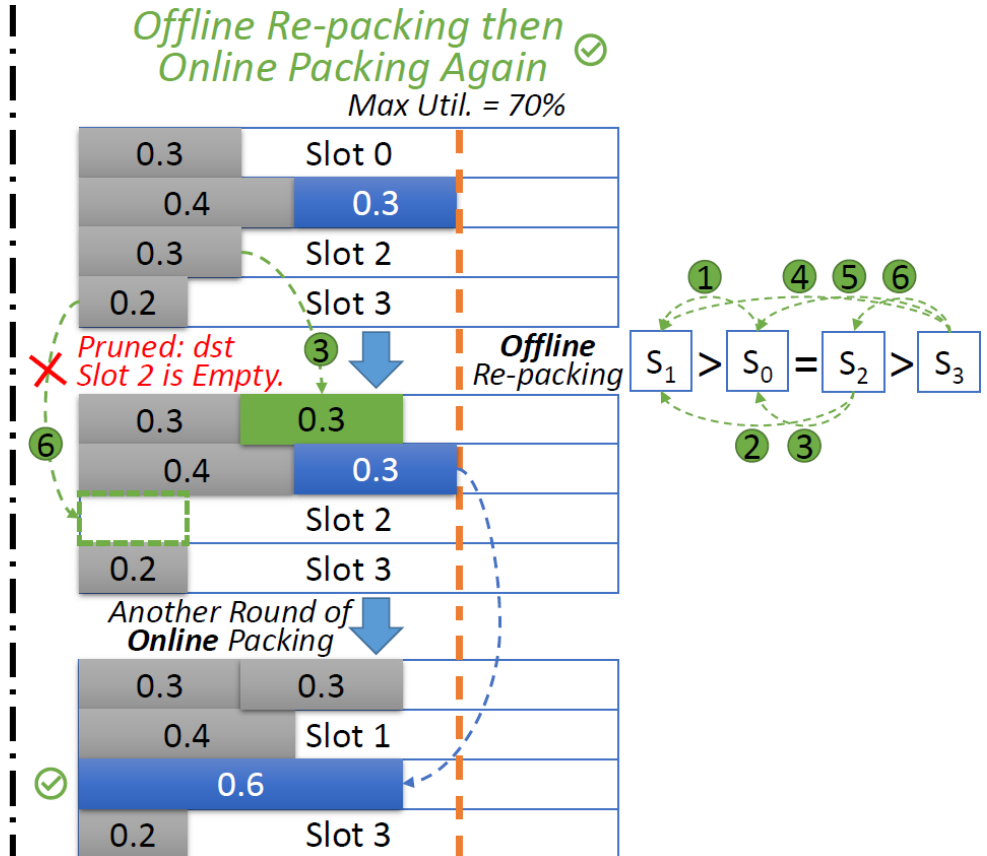
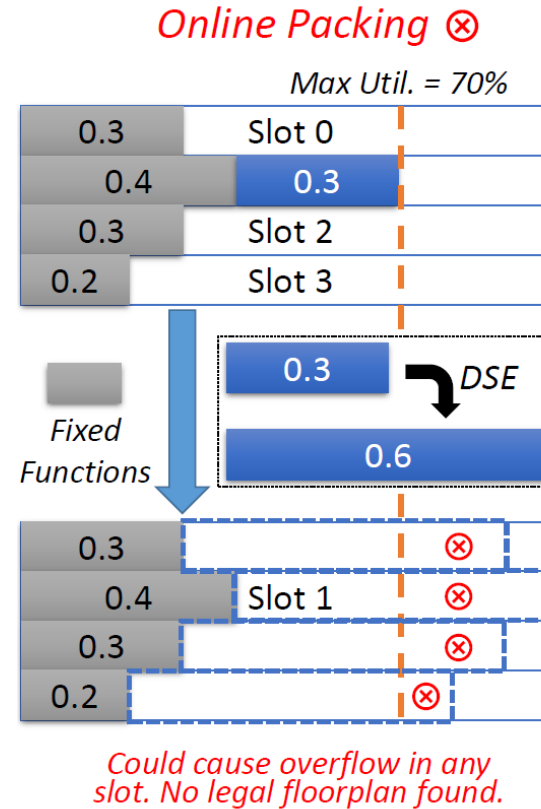
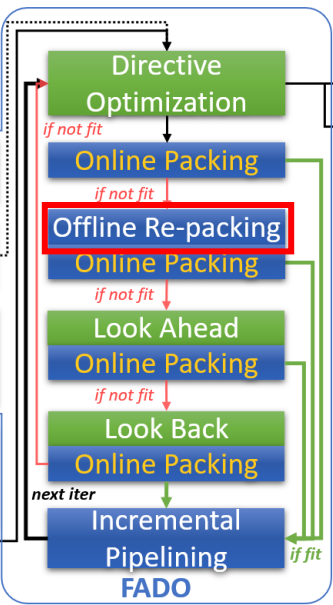
Check SLL util.

Implementation

- Incremental Floorplanning

- Offline Re-packing

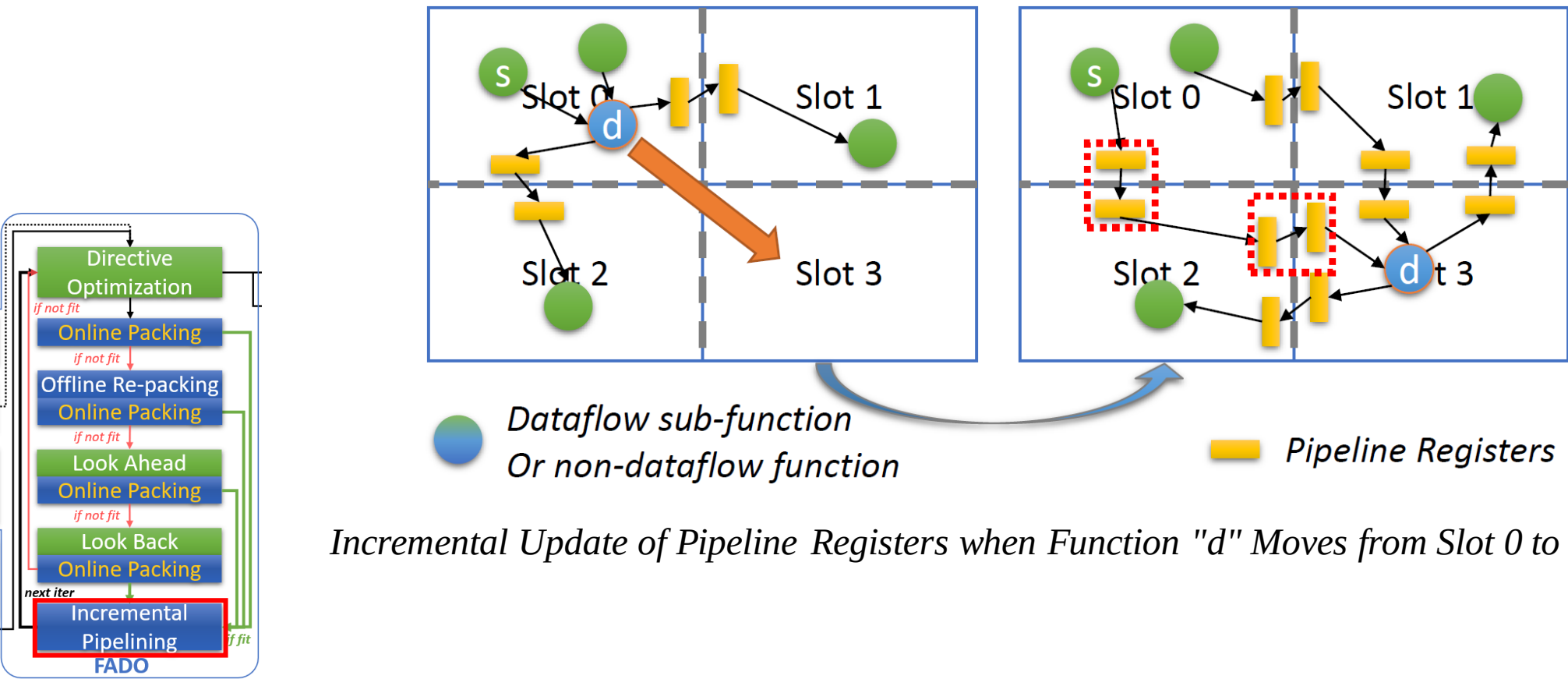
- Best-Fit Decreasing



An Example Showing How Offline Best-Fit Decreasing Re-packing Facilitates Further Directive Search.

Implementation

- Incremental Pipelining
 - Add/Updates/Removes the pipeline registers incrementally.



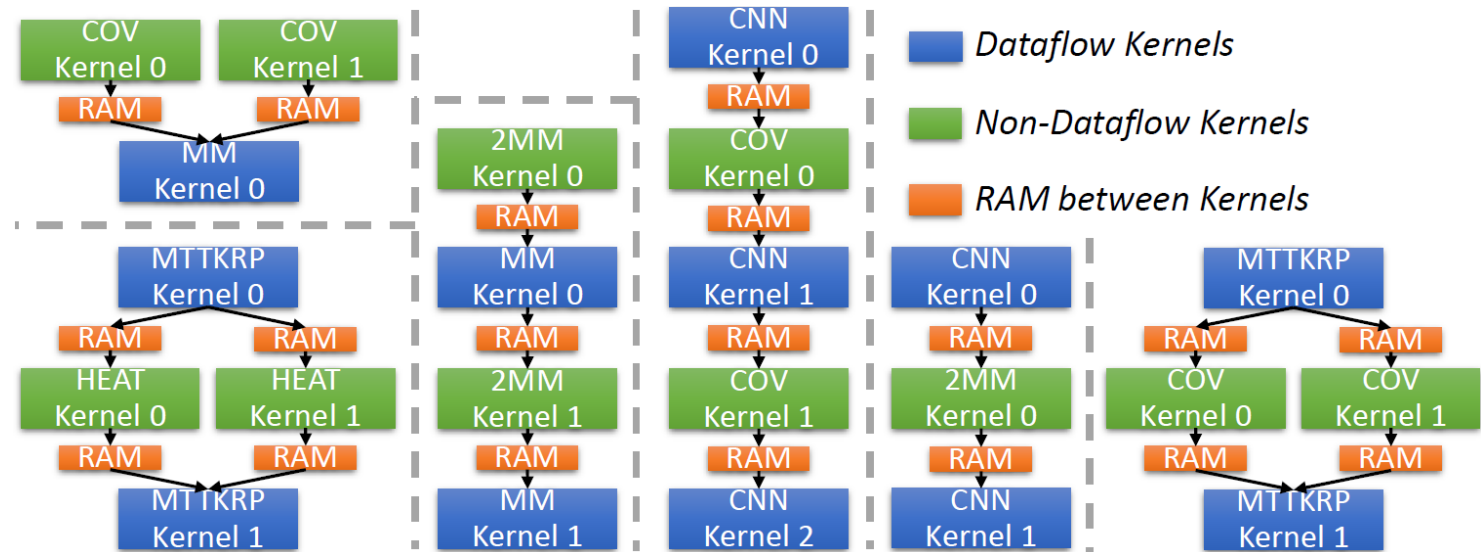
Results

- Experiment Settings
 - Benchmarks
 - Dataflow kernels: from PolySA, AutoSA
 - Non-dataflow kernels: from Polybench

- Tools
 - Vitis HLS/Vitis/Vivado 2020.2

- Board
 - Alveo U250 FPGA

- Resource Constraint
 - 65%



The 6 Benchmarks Assembled to Evaluate FADO

Results

- QoR Comparison between FADO and Other DSE Strategies
 - Runtime speedup: 693X ~ 4925X
 - Overall execution time: 1.16X ~ 8.78X
 - Average Fmax: 290.96 MHz

Benchmarks	CNN*2+2MM*1					MM*1+COV*2					MTTKRP*2+HEAT*2				
	Resource ^a	Runtime ^b	Latency ^c	Fmax ^d	Exe_time ^g	Resource	Runtime	Latency	Fmax	Exe_time	Resource	Runtime	Latency	Fmax	Exe_time
Original (no directive)	28%	-	8933	-	-	20.35%	-	131839	-	-	57.14%	-	8147919	-	-
Initial FP ^e -> Iterative DO ^f	28%	2.24	735	300.45	2,445	19.40%	0.16	131839	282.86	466,094	45.73%	1.36	8138605	<i>Failure</i>	-
Iterative (DO + AutoBridge FP)	48%	1658	92.7	235.89	393	40.66%	10307	2549	278.84	9,141	61.59%	3554	598532	159.97	3,741,525
Iterative (DO + Incr FP) (Ours)	55%	2.39	91.2	269.95	338	41%	2.17	1647	274.47	6,001	63%	1.95	128104	300.45	426,374

Benchmarks	MM*2+2MM*2					CNN*3+COV*2					MTTKRP*2+COV*2				
	Resource	Runtime	Latency	Fmax	Exe_time	Resource	Runtime	Latency	Fmax	Exe_time	Resource	Runtime	Latency	Fmax	Exe_time
Original (no directive)	40.28%	-	259516	-	-	30.95%	-	18130	-	-	38.00%	-	8113234	-	-
Initial FP -> Iterative DO	59.18%	1.13	258842	274.10	944,335	38.89%	2.03	6716	300.45	22,354	41.89%	2.18	8113234	300.45	27,003,607
Iterative (DO + AutoBridge FP)	59.65%	32656	67652	<i>Failure</i>	-	61.70%	8301	1278	222.01	5,754	61.09%	12627	562017	300.45	1,870,585
Iterative (DO + Incr FP) (Ours)	57.53%	6.63	66158	300.00	220,527	63.32%	5.04	1233	300.45	4,105	64.49%	4.89	126921	300.45	422,437

^a The maximum utilization ratio among BRAM, DSP, FF, LUT, and URAM. ^b DSE runtime in seconds. ^c Execution time of HLS designs in number of *thousand* clock cycles.

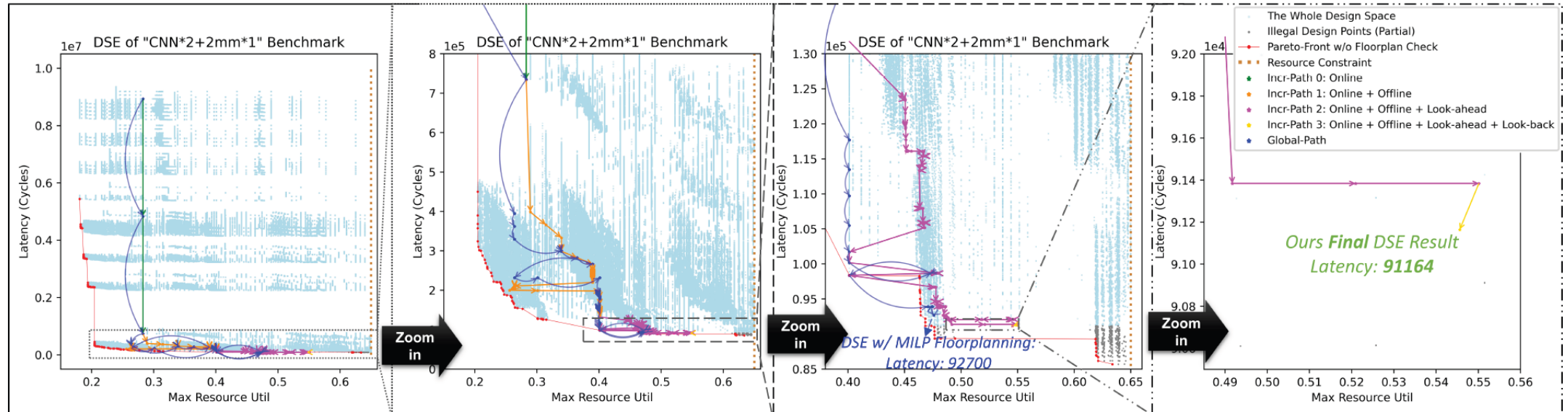
^d Maximum achievable frequency in MHz. ^e FP: Floorplanning. ^f DO: Directive Optimization. ^g Overall execution time (cycle number/frequency) of HLS designs in microseconds (μ s).

Results

• Stages of Floorplan-aware Directive Optimization

Benchmarks	CNN*2+2MM*1		MM*1+COV*2		MTTKRP*2+HEAT*2		MM*2+2MM*2		CNN*3+COV*2		MTTKRP*2+COV*2	
Stages	Resource ^a	Latency ^b	Resource	Latency	Resource	Latency	Resource	Latency	Resource	Latency	Resource	Latency
Online	28.27%	735	40.66%	5167	63.15%	163241	40.28%	259516	31.79%	4718	62.95%	141260
Offline	40.12%	132	40.66%	5167	64.67%	153927	40.28%	259516	31.79%	4718	62.95%	141260
Look-Ahead	55.01%	91.4	40.66%	1651	63.26%	129184	55.25%	66184	31.79%	4718	64.49%	126921
Look-Back	54.56%	91.2	40.66%	1647	63.25%	128104	57.53%	66158	63.32%	1233	64.49%	126921

^a The maximum utilization ratio among BRAM, DSP, FF, LUT, and URAM. ^b Execution time of HLS designs in number of *thousand* clock cycles.

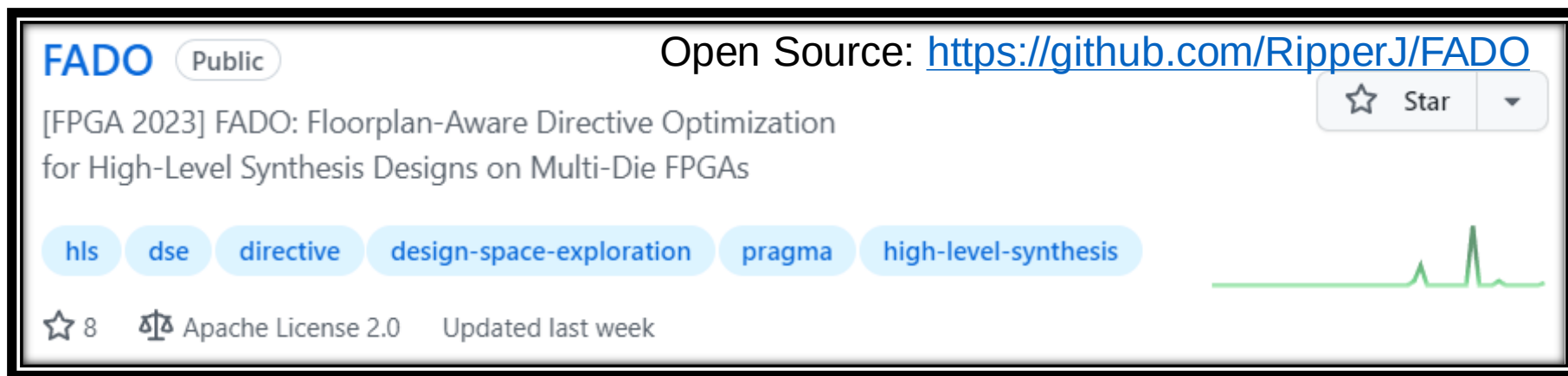


Case Study: DSE Stages and Results on the CNN*2+2MM*1 Benchmark.

Conclusion

- Our contributions:
 - The first solution to co-optimize directive and floorplan of designs on multi-die FPGAs, improving both latency and timing quality in a short runtime.
 - The first precise mathematical formulation of the directive-floorplan co-optimization problem on multi-die FPGAs.
 - 693X ~ 4925X speedup compared with global MILP floorplanning.
 - 1.16X ~ 8.78X shorter overall execution time.
 - Automation of DSE for large-scale applications mixing dataflow/non-dataflow.

Thank you for your attention!



FADO: Floorplan-Aware Directive Optimization
for HLS Designs on Multi-die FPGAs

Linfeng DU

linfeng.du@connect.ust.hk

Reconfigurable Computing Systems Lab, HKUST